
Sistemas Operacionais II

Entrega 6

Anderson Simioni Assunção (23150566)

Bruno Pavani Bertolino (23100464)

Gustavo Fukushima de Salles (21250090)

Julia Fischer Gazolla (23250586)

Arquitetura modificada para P6

- Aumento no número de máquinas virtuais - 4 masters e 8 veículos
- A primeira VM a compor o grupo será atribuída como “master”
- VMs são inicialmente distribuídas em grupos pelo critério de “ID da vm”

GRUPO 0
VM_ID : 0 até 9

GRUPO 1
VM_ID : 10 até 19

GRUPO 4
VM_ID : 20 até 29

GRUPO 3
VM_ID : 30 até 39

Classes modificadas para P6

- Grupos para as VMS em Protocol.h e Gateway.h

```
int coord_of_sender = -1;
if (sender_gid >= 0 && sender_gid < static_cast<int>(GROUP_COORDINATORS.size()))
    coord_of_sender = GROUP_COORDINATORS[sender_gid];

if (my_vm != coord_of_sender) {
    // If we're not the coordinator for this sender's group, we ignore
    printf("[GROUP] Ignoring MOVE_REQUEST from %d for group %d: not my coordination domain (coord=%d, me=%d)\n",
        requester_id, sender_gid, coord_of_sender, my_vm);
    return;
}

// Now we are the coordinator for the sender's group: perform the move locally (thread-safe)
{
    std::lock_guard<std::mutex> lk(groups_mtx);
    // Ensure target group exists, remove from old group (safe even if not present) and add to new group
    _groups[requested_gid].group_id = requested_gid;
    if (_groups.count(sender_gid)) _groups[sender_gid].members.erase(requester_id);
    _groups[requested_gid].members.insert(requester_id);
}
```

```
void on_tick() override {

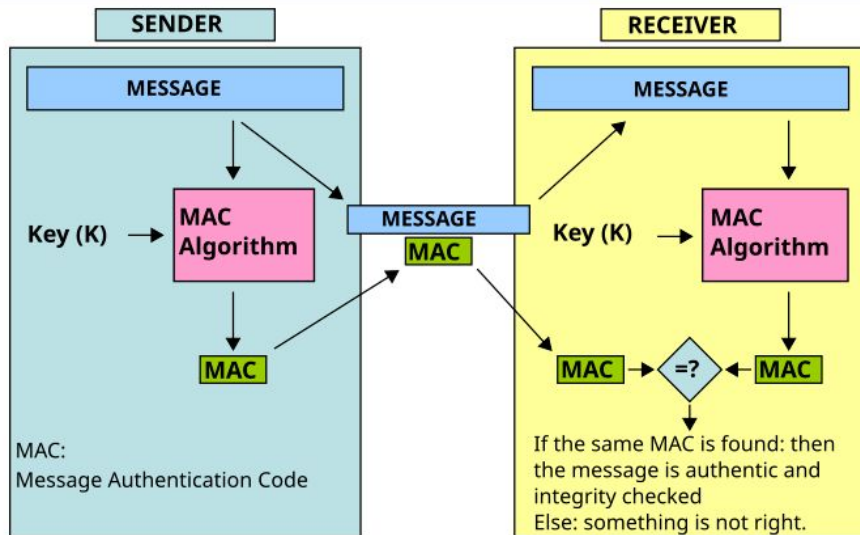
    std::string msg = "PING";
    uint64_t now_us = get_microseconds_now(); // wall-clock timestamp in microseconds

    Endpoint master_endpoint{Ethernet::Address::BROADCAST(), _local.port}; // endpoint

    // GROUP MOVE REQUEST - for testing at the moment
    static bool did_request = false;
    int new_group = 3;
    if(_vm_id == 1 && !did_request) {
        printf("[DEBUG] VM %d requesting to move to group %d.\n", _vm_id, new_group);
        _protocol->move_group(new_group);
        did_request = true;
        printf("[DEBUG] VM %d requested to move to group %d.\n", _vm_id, new_group);
    }
}
```

Verificação de segurança

- Protocol.h



```
uint8_t generate_mac(const char* buf) {  
    size_t len = strlen(buf);  
    uint8_t msgac = 0;  
  
    for (size_t i = 0; i < len; i++) {  
        msgac ^= buf[i] ^ SECRET_KEY[i % SECRET_KEY_LEN];  
    }  
    return msgac;  
}  
  
bool verify_msgac (uint8_t my_msgac, const char* buf ){  
  
    uint8_t expected = generate_mac(buf);  
    printf("[DEBUG AUTH] verify : expected %d and my mac %d \n", expected, my_msgac);  
    return expected == my_msgac;  
}
```

```
static const uint8_t SECRET_KEY[] = { 0x41, 0x23, 0x55, 0x88 }; // chave compartilhada  
static const size_t SECRET_KEY_LEN = sizeof(SECRET_KEY);
```

Verificação de segurança

```
// SYNC CLOCKS PROTOCOL:

else if(ctrl == ControlType::SYNC_REQ) {

    if(_is_master) {
        // Master received SYNC_REQ from worker
        uint64_t t1 = get_microseconds_now(); // master current time

        char buf[128];
        snprintf(buf, sizeof(buf), "%lu", (unsigned long)t1);

        uint8_t msgac = generate_mac(buf);

        // Send SYNC_RESP with timestamp t1
        send_control(_control_local, c.from, ControlType::SYNC_RESP, msgac, buf
```

```
void notify_by_port(const AsyncCapsule& c) {
    Message msg = parse_message(c.payload);
    //printf("[DEBUG] Protocol layer received message [%s] from %s\n", msg.

    // Determine control type
    ControlType ctrl = msg.control;

    // Verify message auth code
    if (ctrl != ControlType::NONE) {
        bool verify_msgac = this->verify_msgac(msg.msgac, msg.body.c_str());
        if (!verify_msgac)
        {
            //printf("[AUTH] Unauthorized control message! \n");
            return;
        }
    }
}
```

Message

- Estrutura final da mensagem

```
9  };
10
11  struct Message {
12      uint8_t  group_id{0};           // Road segment of the original sender VM (from first sender)
13      uint8_t  orig_vm{0};           // Original sender VM
14      uint16_t orig_port{0};         // Original sender port
15      uint64_t timestamp{0};         // Original TIME
16      int      type{0};              // Message type (like component port)
17      uint64_t msg_id{0};            // Original ID
18      uint8_t  msgac{0};             // Message auth code
19      std::string body;              // Payload body
20      ControlType control{ControlType::NONE}; // Control type, default NONE
21  };
22
23  static std::string build_message(const Message& msg) {
24      std::ostringstream oss;
25      oss << msg.group_id << " " << msg.orig_vm << " " << msg.orig_port << " " << msg.timestamp << " " << msg.type << " " << msg.msg_id << " " << msg.msgac << " " << msg.body << " " << msg.control << "\n";
26      return oss.str();
27  }
```

Dados de latência - 12 VMs

```
latency_reports >  summary_latency_per_vm.csv
1  recv_vm,recv_port,count,avg_us,median_us,std_us
2  0,65535,86,"464,747.38","754,717.50","445,736.62"
3  1,102,37,"5,379.95","4,116.00","4,293.08"
4  1,65535,112,"205,612.17","4,125.00","558,068.74"
5  2,102,4,"12,627.50","13,483.50","4,770.89"
6  2,65535,21,"38,056.52","14,987.00","47,835.11"
7  10,65535,6,"196,115.33","180,233.50","213,388.61"
8  11,65535,5,"208,074.40","11,974.00","315,036.94"
9  12,102,7,"7,768.00","4,634.00","8,571.85"
10 12,65535,10,"196,405.40","4,257.50","609,524.66"
11 20,65535,7,"12,038.43","3,705.00","22,197.80"
12 21,102,4,"12,586.75","9,546.50","7,234.90"
13 21,65535,23,"82,983.39","5,533.00","107,112.63"
14 22,65535,4,"4,701.75","4,082.50","3,380.83"
15 30,65535,5,"334,100.00","524,433.00","301,154.90"
16 31,102,1,"13,878.00","13,878.00",0.00
17 31,65535,3,"1,902.00","1,320.00","1,103.80"
18 32,65535,2,"13,369.00","13,369.00",169.71
19
```

Pontos gerais do projeto

```
[BUILD] Compilando para x86_64...  
g++ build/x86/car.o build/x86/engine_ethernet.o  
tatic  
rm -rf build  
[CLEAN] Diretório build/ limpo.  
[CPIO] Gerando initramfs...  
10160 blocks  
[CPIO] initramfs.cpio gerado com sucesso.
```

```
[INFO] Starting VM 31 with MAC 52:54:00:12:34:1f, logging to /home/julia/...  
[INFO] Timeout set to 80s - VMs will be terminated after this period.  
[INFO] Timeout reached - terminating VMs  
[INFO] Cleaning up VMs...  
[INFO] All VMs have finished or were terminated.  
[INFO] Logs saved in /home/julia/Documents/S02/S02P6/S02-project/logs  
[INFO] Cleaning up VMs...  
[ANALYZE] Calculando latências a partir dos logs...  
[INFO] Scanning logs in /home/julia/Documents/S02/S02P6/S02-project/logs  
[INFO] Detailed CSV for VM 11 saved to /home/julia/Documents/S02/S02P6/S02...  
[INFO] Detailed CSV for VM 10 saved to /home/julia/Documents/S02/S02P6/S02...  
[INFO] Detailed CSV for VM 31 saved to /home/julia/Documents/S02/S02P6/S02...
```

- Uso otimizado das bibliotecas do busybox (10160 blocos), mantendo no repositório apenas o necessário;
 - Geração de arquivos de logs e análise de latência através de algoritmo em python;
 - Uso de x86, evitando compilação cruzada;
 - Todas as máquinas virtuais encerram após tempo determinado (80s);
 - Análise de tráfego com wireshark.
-

Diferenciais da Arquitetura

- A sincronização de barreira de inicialização de vms e sincronização temporal ocorre através do mesmo mecanismo: mensagens de controle

```
// Message type: control or data
enum class ControlType {
    NONE,    // Not a control message
    READY,
    GO,
    SYNC_REQ,
    SYNC_RESP,
    DELAY_REQ,
    DELAY_RESP,
    GROUP_REGISTER,
    GROUP_MOVE_REQUEST,
    GROUP_MOVE_NOTIFY,
    GROUP_ASSIGN
};
```

Diferenciais da Arquitetura

- Gateway é um componente como os outros, mas é o único que faz comunicação externa da VM
- VMs master e veículos têm gateway. Os demais componentes estão apenas nos veículos

```
template <typename TNIC>
class Gateway : public CarComponent<TNIC> {
    using Base = CarComponent<TNIC>;
    using Base::_comm;
    using Base::_protocol;
    using Base::_local;
    using Endpoint = typename Base::Proto::Endpoint;
    using Rx      = typename Base::CommunicatorT::Rx;
    using Port = typename Protocol<TNIC>::Port;
```

```
void Car::start(int vm_id, int group_id, int total_sync_vms)
{
    int components_len = this->components.size();

    // Determine VM identity
    bool is_sync_master = ((vm_id % 10) == 0); // VMs 0, 10, 20, ...
    if (is_sync_master) components_len = 0;
```

Diferenciais da Arquitetura

- O shm_node possibilita sincronização de processos para gerenciar o uso da memória compartilhada entre os componentes.

```
void* ShmNode::initialize_shared_memory_region()
{
    //printf("creating/opening shared memory region..\n");
    int shm_id = shm_open(this->shared_memory_region_name, O_CREAT | O_RDWR, S_IRUSR | S_IWUSR);

    if(ftruncate(shm_id, sizeof(SharedData)) == -1)
    {
        //printf("error on create/open shared memory region!\n");
        exit(-1);
    }

    //printf("mapping shared memory region..\n");
    this->shared_data_ptr = (SharedData*)mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE, MAP_SHARED, shm_id, 0);
    if (this->shared_data_ptr == MAP_FAILED) exit(-1);

    if(this->is_master_node)
    {
        memset(this->shared_data_ptr, 0, sizeof(SharedData));
        this->shared_data_ptr->parent_pid = getpid();
    }

    return this->shared_data_ptr;
}
```

Diferenciais da Arquitetura

- Publish-subscribe

```
class PublishSubscriber {
public:
    using SubscriberId = std::string;
    using u64 = std::uint64_t;

    void add_capability(VehicleDataType type) { capabilities_.insert(type); }
    void remove_capability(VehicleDataType type) { capabilities_.erase(type); }
    // tell what data types this component can produce
    bool supports(VehicleDataType type) const { return capabilities_.count(type) > 0; }
```

```
bool digest_subs(const Rx& rx) {
    std::string s = rx.msg.body;
    size_t p1 = s.find("SUBS=");
    size_t p2 = s.find("SUBS_TYPE=");
    size_t p3 = s.find("SUBS_PERIOD=");
    if (p1 == std::string::npos || p2 == std::string::npos || p3 == std::string::npos) return false;

    size_t e1 = s.find(' ', p1); if (e1 == std::string::npos) e1 = s.size();
    size_t e2 = s.find(' ', p2); if (e2 == std::string::npos) e2 = s.size();
    size_t e3 = s.find(' ', p3); if (e3 == std::string::npos) e3 = s.size();

    std::string addr = s.substr(p1 + 5, e1 - (p1 + 5));
    VehicleDataType dtype = static_cast<VehicleDataType> (
        std::stoi(s.substr(p2 + 10, e2 - (p2 + 10)))
    );
    uint64_t period = std::stoull(s.substr(p3 + 12, e3 - (p3 + 12)));

    if(!_pubSub.supports(dtype)) _pubSub.subscribe(dtype, addr, period);

    return true;
}
```

Diferenciais da Arquitetura

```
// SYNC CLOCKS PROTOCOL:
else if(ctrl == ControlType::SYNC_REQ && is_message_internal(msg)) {

    if(_is_master) {
        // Master received SYNC_REQ from worker
        uint64_t t1 = get_microseconds_now(); // master current time

        char buf[128];
        snprintf(buf, sizeof(buf), "%lu", (unsigned long)t1);

        uint8_t msgac = generate_mac(buf);

        // Send SYNC_RESP with timestamp t1
        send_control(_control_local, c.from, ControlType::SYNC_RESP, msgac, buf);

        //printf("[SYNC] SYNC_REQ received from %s, sending SYNC_RESP with t1=%llu\n", c.from.mac.str().c_str(), t1);
    }

}

else if(ctrl == ControlType::SYNC_RESP && is_message_internal(msg) && !_is_master) {
    // Worker received SYNC from master
    uint64_t t1 = 0;
    if(!msg.body.empty()) sscanf(msg.body.c_str(), "%lu", (unsigned long*)&t1);

    uint64_t t2 = get_microseconds_now(); // worker receive time
    uint64_t t3 = t2; // worker send time, the same for now

    char buf[256];
    snprintf(buf, sizeof(buf), "%lu %lu %lu", (unsigned long)t1, (unsigned long)t2, (unsigned long)t3);

    uint8_t msgac = generate_mac(buf);

    send_control(_control_local, c.from, ControlType::DELAY_REQ, msgac, buf);

    //printf("[SYNC] Received SYNC_RESP from %s (t1=%llu), sending DELAY_REQ (t2=%llu, t3=%llu)\n", c.from.mac.str().c_str(), t1, t2, t3);
}

}
```

- Sincronização temporal no Protocol.h

Diferenciais da Arquitetura

- Sincronização temporal no Gateway.h:

```
// SYNC REQUEST every 3 seconds
if (!_sync_master) {
    if (now_us - _last_sync_request_us.load() >= 3'000'000ULL) { // 3 seconds
        if(_protocol->get_running_ptp())
        {
            _protocol->set_probabilistic_ptp_timeout(true);
        }
        else
        {
            _protocol->set_running_ptp(true);
            _protocol->set_probabilistic_ptp_timeout(false);
            _last_sync_request_us.store(now_us);

            //printf("[SYNC][VM %d] Sending SYNC_REQ to master\n", _vm_id);

            _protocol->send_control(_local, master_endpoint, Protocol<TNIC>::ControlType::SYNC_REQ);
        }
    }
}
```

Diferenciais da Arquitetura

- Sincronização temporal - clock_syncer.h

```
// Z-values for Normal Curve
// 99.99% = 3.891
// 99.00% = 2.575
// 97.50% = 2.2414
// 95.00% = 1.96
// maxWidthUs = max total width of the interval you accept, in microseconds, e.g.: 2.0 means average  $\pm 1$  microseconds
// Build a confidence interval around the mean: mean  $\pm z * SE$  and look at the total width:  $2 * z * SE$ 
bool hasEnoughSamplesCI(long long maxWidthUs, double confidenceZ = 3.891) const
{
    size_t n = offset_samples.size();
    if(n >= MAX_SAMPLES_IN_CLOCK_SYNCER) return true;
    if(n < 30) return false; // keep collecting, minimum of 30 data points should guarantee a representative sample
    // But also check whether we have enough samples according to the selected Z
    long long se = getOffsetStdError(); // standard error of the mean
    long long width = (long long)2 * se * confidenceZ; // total width of the CI

    // Return if the interval is good enough
    return width <= maxWidthUs;
}
```

Conclusões

- De forma geral, a equipe conseguiu concluir as tarefas propostas para a sexta entrega do projeto de Comunicação em Sistemas Autônomos Críticos.
 - Foi possível implementar camada de segurança entre as mensagens de controle e separar as máquinas virtuais em grupos.
-